CareerOS

AI-Powered Job Application Tracker & Resume Intelligence Platform

Technical Project Report | Version 3 | November 2026

Executive Summary

CareerOS is a local-first full-stack application that combines job application tracking, authenticated APIs, PDF resume analysis, explainable keyword matching, pipeline analytics and optional AI-generated feedback. The current local integration uses Ollama with Llama 3.2 3B. It is a working development prototype, not a production deployment.

1. Problem and objectives

Job seekers manage multiple applications and repeatedly compare resumes against lengthy postings. CareerOS centralizes application status, exposes progress metrics, and provides a transparent first-pass view of skill overlap. Objectives include usability, authenticated user separation, explainable matching and a local AI option.

2. Technology stack

Frontend: React 19, Vite 6, JavaScript, CSS, Lucide React and Recharts. Backend: Python 3.11, FastAPI, Pydantic, SQLAlchemy and JWT-based authentication. Storage: SQLite locally, with PostgreSQL-oriented configuration. Resume parsing: pypdf. Testing: pytest. AI: Ollama serving llama3.2:3b.

3. Architecture

The browser accesses the React development server at port 5173. Vite proxies API requests to FastAPI at port 8000. The API accesses the SQL database, runs PDF parsing and deterministic matching, and optionally sends structured skill lists and a job description to the Ollama HTTP service at port 11434. Ollama runs as a separate local process.

4. Implemented functionality

Registration and login; per-user job application create/read/update/delete; pagination and search; status analytics; CSV export; saved skill profile; job-description keyword matching; PDF upload/extraction; resume-to-job comparison; optional AI feedback; OpenAPI docs and backend tests.

5. Data and security

The SQLAlchemy models are User, Application and Skill. Application and Skill records belong to users. Passwords are hashed and protected endpoints require JWT authentication. Resume PDFs are processed in memory and are not saved. Development secrets and local databases must not be committed. Production hardening remains outstanding.

6. Resume comparison methodology

The application extracts text from a non-encrypted text PDF (maximum 2 MB, 12 pages), detects explicit skill phrases using a fixed catalog and regular expressions, and compares those skills against phrases found in a job description. Coverage equals matched detected job skills divided by all detected job skills, multiplied by 100. This is not an ATS prediction or a measure of actual proficiency.

7. Local AI workflow

FastAPI sends the job description and matched/missing skill arrays to the local Llama model through Ollama /api/chat. The model returns a short advisory text. No model training or fine-tuning was performed. The model does not receive the full resume text through this endpoint and can produce incorrect or contradictory suggestions.

8. Development history

Phase 1: FastAPI/SQLite backend, plain HTML UI, CRUD, JWT, analytics, CSV and tests. Phase 2: PDF parsing and transparent skill matching. Phase 3: React/Vite frontend and AI endpoint. Phase 4: Ollama integration on Apple Silicon. The project owner observed successful HTTP 200 responses for login, analytics, export, resume analysis, comparison and AI feedback.

9. Testing and observations

Earlier backend tests passed in local development. User-provided logs confirm successful requests to key endpoints, including POST /api/ai/feedback returning HTTP 200. A repeated registration returned HTTP 409, consistent with an existing account. These observations do not establish production reliability, model accuracy, or full frontend test coverage.

10. Known limitations

Keyword matching does not understand context, synonyms or required versus preferred qualifications. Scanned PDFs are unsupported. The local LLM sometimes contradicts deterministic matches: one observed response called React and TypeScript missing despite their presence in the matched list. AI output must be checked. The frontend could benefit from component refactoring and broader automated tests.

11. Reproduction instructions

Install Python 3.11+, Node/npm and Ollama. Create a Python virtual environment; install requirements-dev.txt. In frontend/, run npm install. Pull llama3.2:3b. Run ollama serve, uvicorn app.main:app --reload --port 8000, and npm run dev in frontend/ in three separate terminals. Visit http://127.0.0.1:5173. Run python -m pytest -q and npm run build to validate.

12. Future work

Refactor React into components, improve semantic and contextual matching, enforce consistency checks on AI advice, add end-to-end tests, improve accessibility, harden authentication and deploy using a documented production architecture.

Conclusion

CareerOS demonstrates integration across UI development, REST APIs, database persistence, document processing, explainable scoring and local LLM inference. It should be presented as a functional local prototype with documented limitations, rather than an ATS predictor or production service.